FACULTÉ DES SCIENCES ET TECHNOLOGIES | Université de Lille

Documentation Technique

Application SpaceX Tracker



Auteur : Bouchra Bouamama

Table des matières

1 Introduction **3**
1.1 Contexte du projet 3
1.2 Objectif de l'application 3

2 Mode d'emploi et Compatibilité **4**
2.1 Installation 4
2.2 Compatibilité 4
2.3 Navigation principale 4

3 Les Choix Techniques avec Explication des Choix **5**
3.1 Pile technologique 5

4 Librairies Utilisées **6**
4.1 Retrofit (com.squareup.retrofit2 :retrofit :2.11.0) 6
4.2 Moshi (com.squareup.moshi :moshi-kotlin :1.15.1) 6
4.3 Coil (io.coil-kt :coil-compose :2.7.0) 6
4.4 Kotlinx-Coroutines (org.jetbrains.kotlinx :kotlinx-coroutines-android :1.8.1) 7

5 Concepts Métier et Gestion des Données **8**
5.1 Entités principales 8
5.2 Règles Métier 8

6 Contrat d'Interface API **9**
6.1 Authentification 9
6.2 Les Requêtes 9
6.3 Format des Données 9
6.4 Gestion des Erreurs API 10
6.5 Pourquoi ce choix d'API ? 11

7 Diagrammes de classes et packages **12**

8 Diagrammes de séquences **14**
8.1 Flux Nominal : Récupération Online et Affichage 14
8.2 Résilience : Scénario Offline avec Fallback 15

9 Architecture et Fonctionnement **16**
9.1 Organisation en couches 16
9.2 Parcours d'un flux de données (Data Flow) 17
9.3 Librairies tierces 17
9.4 Parc Matériel et Cible 18
9.5 Éléments clés de compréhension 18

10 Améliorations et Perspectives **19**

11 Conclusion **20**

12 Bibliographie / Webographie **21**

Chapitre 1

Introduction

1.1 Contexte du projet

Ce projet a été réalisé dans le cadre de l'Unité d'Enseignement *Développement Android Mobile* du Master 1 E-Services. Il vise à appliquer les bonnes pratiques du développement Android moderne, tant sur le plan architectural que technique.

L'application développée, SpaceX Tracker, permet de consulter les lancements de la société SpaceX à partir de données officielles exposées par une API publique.

1.2 Objectif de l'application

L'objectif principal est de fournir une interface intuitive et performante permettant de consulter l'historique et le calendrier des missions spatiales. L'app démontre les bonnes pratiques d'intégration API dans Android.

Chapitre 2

Mode d'emploi et Compatibilité

2.1 Installation

L'application est fournie sous forme de package Android (APK). Elle peut également être compilée et déployée via Android Studio à l'aide du système de build Gradle : Cloner le dépôt GIT, ouvrir dans Android Studio (version Koala ou supérieure), et build/run sur un émulateur ou device physique.

2.2 Compatibilité

- **Version Android minimale :** API 26 (Android 8.0)
- **Version cible :** API 35 (Android 15)
- **Orientation :** Portrait et Paysage

2.3 Navigation principale

- Écran Principal : La liste des lancements SpaceX. Chaque item montre le nom, la date formatée, le statut (succès/échec), l'image du patch avec recherche plein texte
- Scroll : Défilez vers le bas pour voir tous les lancements (200 items)
- Gestion des favoris : Les lancement marqués comme favoris s'afficheront dans un écran dédié.
- Écran du détail : Vue détaillée d'un lancement
- Gestion des Erreurs : Si pas de réseau, passage en mode hors-ligne automatique
- Rotation d'Écran : L'app s'adapte automatiquement : en paysage, la grille passe à deux colonnes par ligne.
- Aucune authentification requise ; l'API est publique

Chapitre 3

Les Choix Techniques avec Explication des Choix

3.1 Pile technologique

— **Architecture MVVM** : Utilisée pour séparer l'UI (Compose) de la logique métier (ViewModel) et des données (Repository). **Avantage** : testabilité, maintenance, et réactivité avec StateFlow/MutableState.

— **Jetpack Compose** : Choisi pour l'UI déclarative, plus moderne que XML/Views. Permet une adaptation facile à la rotation via LocalConfiguration.

— **Retrofit + Moshi + Coroutines** : Pour les appels API, Retrofit simplifie les requêtes HTTP, Moshi gère le JSON, Coroutines pour l'asynchronicité sans callbacks complexes.

— **Room** : Persistance locale SQLite

— **Coil** : Pour le chargement d'images, compatible Compose, léger, avec caching intégré. Préféré à Glide pour son intégration native avec Compose et sa simplicité.

— **ViewModelScope** : Pour gérer les coroutines liées au lifecycle, évitant les leaks.

— **LazyColumn** : Pour un affichage performant de listes longues, avec chargement lazy. Scrollbar custom ajoutée pour visibilité.

— **Rotation** : Gérée via LocalConfiguration.current.orientation, sans configChanges dans Manifest pour laisser Compose recomposer l'UI. État persisté via ViewModel.

Ces choix assurent une app performante, scalable, et conforme aux standards Android modernes.

Chapitre 4

Librairies Utilisées

4.1 Retrofit (com.squareup.retrofit2 :retrofit :2.11.0)

— **Pourquoi cette librairie** : Pour simplifier les appels HTTP à l'API SpaceX. Elle gère les endpoints, paramètres, et conversions automatiques.
— **Avantages/Inconvénients** : **Avantages** : Typage fort, intégration avec converters (Moshi), support coroutines. **Inconvénients** : Courbe d'apprentissage initiale comparé à HttpURLConnection, mais plus efficace que Ktor pour Android.
— **Ce qu'elle fait** : Crée des interfaces API (e.g., @GET("v5/launches")), exécute les requêtes, et retourne des objets Kotlin.

4.2 Moshi (com.squareup.moshi :moshi-kotlin :1.15.1)

— **Pourquoi cette librairie** : Bibliothèque optimisée pour Kotlin, offrant une meilleure gestion des valeurs nullables et des performances supérieures à Gson lors du parsing JSON.
— **Avantages/Inconvénients** : **Avantages** : Rapide, annotations simples (@Json). **Inconvénients** : Moins features que Gson, mais suffisant ici. Préféré à Gson pour sa performance.
— **Ce qu'elle fait** : Convertit JSON en objets Launch via adapter factory.

4.3 Coil (io.coil-kt :coil-compose :2.7.0)

— **Pourquoi cette librairie** : Chargeur d'images asynchrone conçu pour Kotlin et Jetpack Compose, avec une gestion optimisée du cache mémoire et disque.
— **Avantages/Inconvénients** : **Avantages** : Intégration native Compose, caching, placeholders. **Inconvénients** : Moins mature que Glide, mais plus simple.
— **Ce qu'elle fait** : Charge images depuis URL avec AsyncImage, gère erreurs/null.

4.4 Kotlinx-Coroutines (org.jetbrains.kotlinx :kotlinx-coroutines-android :1.8.1)

— **Pourquoi cette librairie** : Gestion asynchrone pour API calls.

— **Avantages/Inconvénients** : **Avantages** : Structured concurrency, facile avec viewModelScope. **Inconvénients** : Besoin de gérer exceptions. Préféré à RxJava pour sa simplicité Kotlin-native.

— **Ce qu'elle fait** : Lance suspend functions pour fetch data sans bloquer UI.

Autres : androidx.lifecycle pour ViewModel, compose-bom pour UI.

Chapitre 5

Concepts Métier et Gestion des Données

5.1 Entités principales

— **Launch** : mission SpaceX (nom, date, succès, médias)
— **Préférences utilisateur** : favoris et filtres persistants

5.2 Règles Métier

— Fetch des lancements depuis API.
— Les favoris sont persistés localement
— Single Source of Truth : L'application applique le principe de *Single Source of Truth*. Toutes les données sont stockées dans la base locale Room avant d'être affichées, garantissant une cohérence même en cas de perte réseau.

Chapitre 6

Contrat d'Interface API

L'application communique avec l'API publique de SpaceX, open-source, sans authentification. Toutes les requêtes sont effectuées en GET et les données sont formatées en JSON.

6.1 Authentification

Aucune requête d'authentification. Rate limit non-officiel 300 requêtes/heure/IP (basé sur communauté).

6.2 Les Requêtes

— Endpoint : GET /v4/launches
— Paramètres (Query) :
  — **search** (String) : Recherche par nom/mission.
  — **mode** (String) : "list" ou "detailed" (défaut detailed).
  — **limit** (Int) : Nombre max de résultats (défaut illimité).
  — **offset** (Int) : Offset pour pagination.
  — **Exemple** : https ://api.spacexdata.com/v4/launches ?limit=10&search=falcon

6.3 Format des Données

Réponse : Array JSON de launches. Champs clés (annotés) :
— id (String, obligatoire) : ID unique.
— name (String, obligatoire) : Nom du lancement.
— date_utc (String, obligatoire) : Date UTC (ISO format).
— flight_number (Int, Optionnel) : Numéro de vol.
— success (Boolean, optionnel) : Succès du lancement.

— details (String, Optionnel) : Description textuelle.

— links (Object, optionnel) : patch small (String, optionnel) : URL image, large (String, optionnel) : URL image, article (String, Optionnel) : Lien vers l'article .

Exemple de réponse JSON (Objet Launch)

```
{
  "id": "5eb87cd9ffd86e000604b32a",
  "name": "FalconSat",
  "date_utc": "2020-03-06T05:50:31.000Z",
  "flight_number": 1,
  "success": true,
  "details": "Engine failure at T+33s",
  "links": {
    "patch": {
      "small": "https://images2.imgbox.com/3c/0d/9681v43f_o.png",
      "large": "https://images2.imgbox.com/40/e3/G8u3yS7X_o.png"
    },
    "article": "https://www.spacex.com/news/..."
  }
}
```

Annotations et Caractère Optionnel

L'API SpaceX est très permissive sur les valeurs nulles (nullable). Dans notre code Kotlin, cela a été géré de la manière suivante :

success : Boolean ? : Le point d'interrogation est vital car pour les missions futures, le champ success n'existe pas encore dans la réponse JSON.

details : String ? : Toutes les missions n'ont pas forcément de résumé textuel disponible.

links : Les URLs de patch peuvent être absentes pour les lancements très anciens ou très récents. Une URL de fallback est utilisée dans le composable LaunchImage.

6.4 Gestion des Erreurs API

L'application traite les codes de retour HTTP standard via Retrofit :

— **200 OK** : Données récupérées avec succès. Déclenche l'insertion dans Room via dao.insertAll().

— **404 Not Found** : Identifiant de vol incorrect.

— **5xx Server Error** : Problème côté SpaceX. L'application bascule alors sur l'affichage des données en cache local (Offline Fallback).

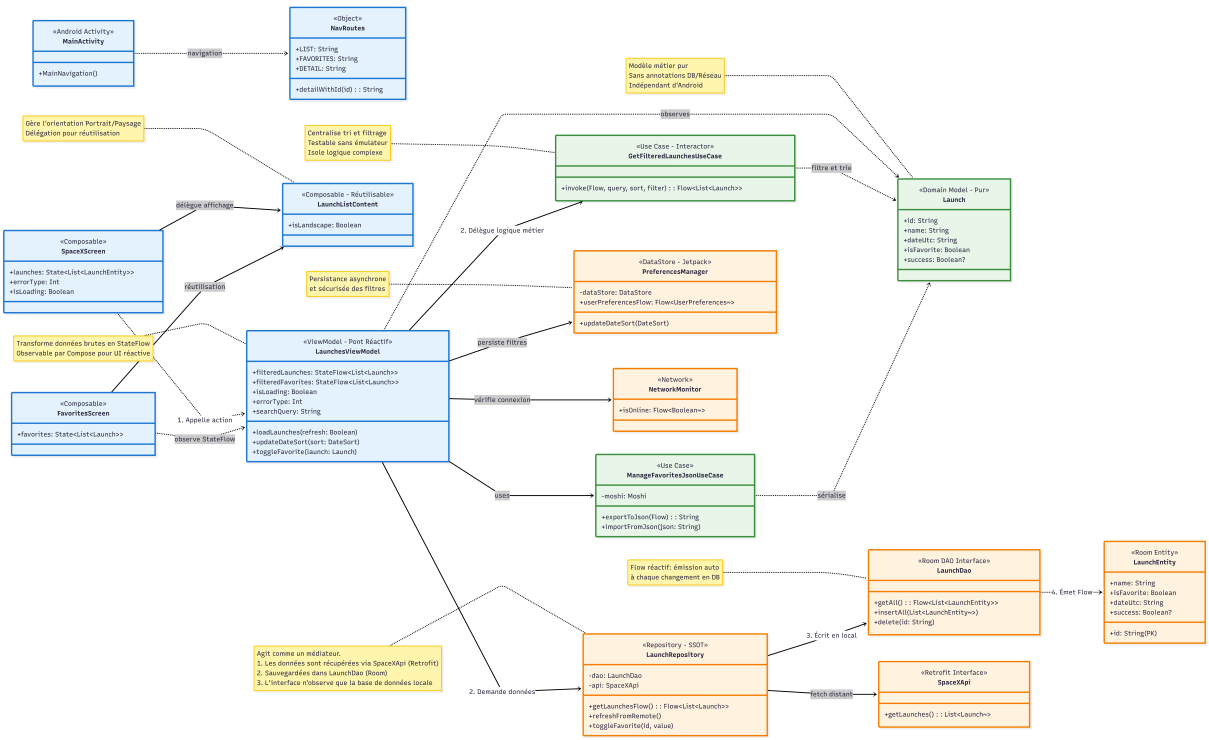
6.5 Pourquoi ce choix d'API ?

L'utilisation de la version v4 de l'API SpaceX permet d'avoir des objets plus légers et mieux structurés que les versions précédentes. En récupérant la liste complète en une fois, nous optimisons la batterie de l'utilisateur : nous ne faisons qu'un seul appel réseau, puis tous les tris et recherches (SearchQuery) se font localement en mémoire via le GetFilteredLaunchesUseCase, ce qui rend l'interface extrêmement fluide.

Chapitre 7

Diagrammes de classes et packages

L'architecture est découpée en packages suivant la Clean Architecture pour éviter les dépendances cycliques.



Le diagramme ci-dessus illustre la séparation stricte des responsabilités au sein de l'application :

Couche UI (Bleu) : Organisée autour du pattern MVVM. Le LaunchesViewModel agit comme un pivot réactif, exposant des StateFlow pour assurer une mise à jour fluide de l'interface Jetpack Compose.

Couche Domaine (Vert) : Contient la logique métier "pure". Les Use Cases (GetFilteredLaunchesUseCase) isolent les algorithmes de tri et de filtrage, garantissant que la

logique reste testable indépendamment d'Android.

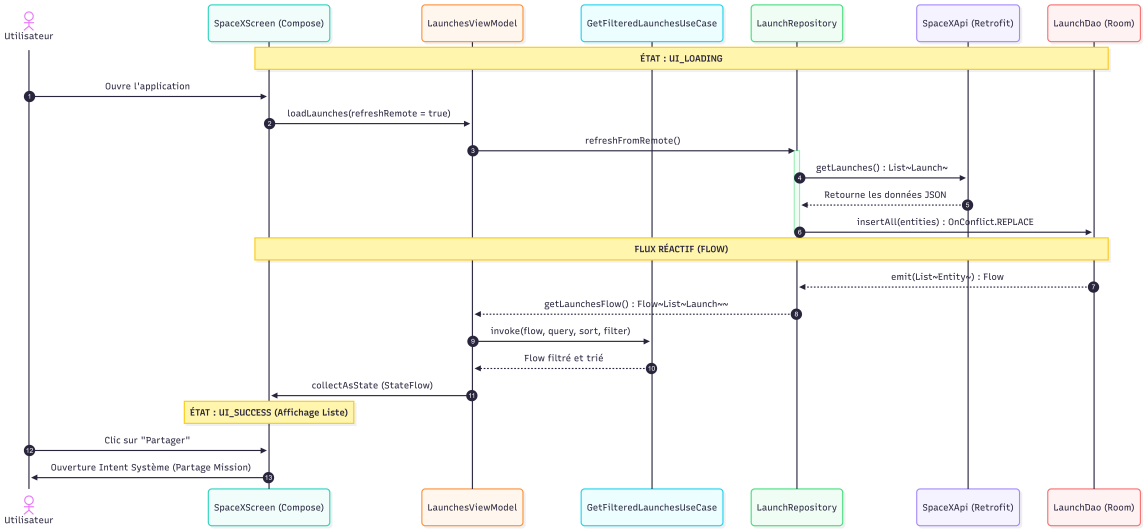
Couche Data (Orange) : Implémente le pattern Repository. Le système garantit une "Source unique de vérité" (SSOT) : les données transitent par l'API (Retrofit) avant d'être persistées dans Room, qui émet ensuite un flux de données (Flow) vers les couches supérieures.

Chapitre 8

Diagrammes de séquences

8.1 Flux Nominal : Récupération Online et Affichage

Ce flux montre la priorité donnée à la base de données locale (SSOT).



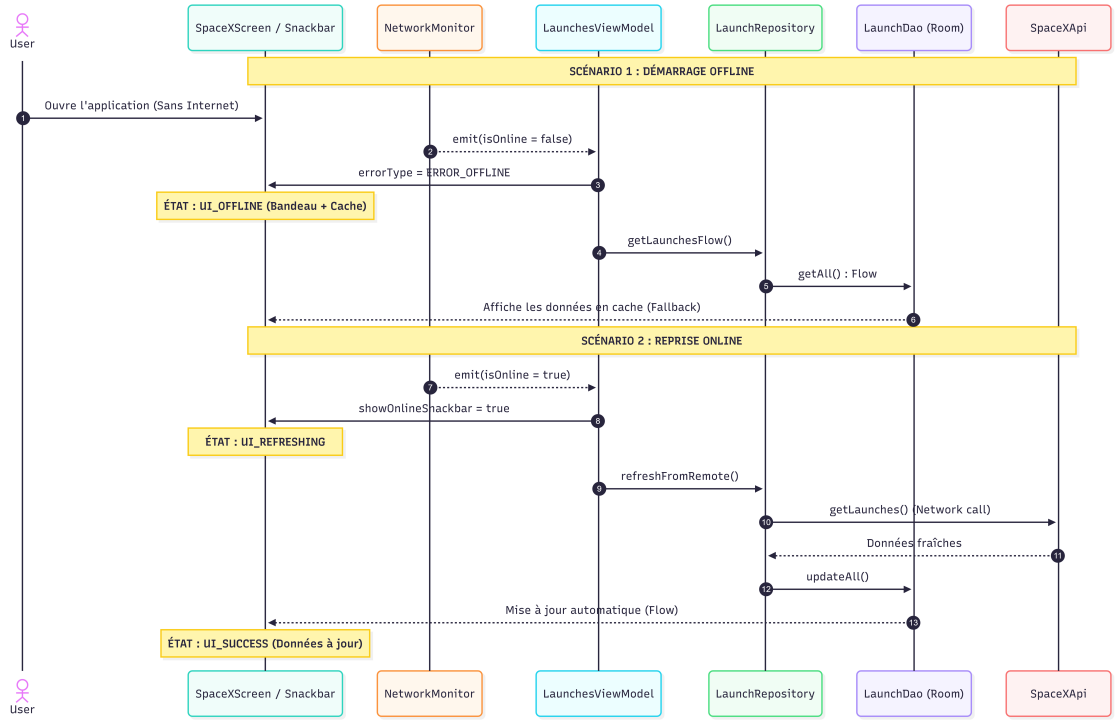
Mécanisme de Cache-First : On remarque que le ViewModel n'attend pas la réponse API pour afficher quelque chose. Il observe un Flow provenant de Room. Dès que l'API écrit dans la DB (étape 5), l'UI se met à jour automatiquement.

Logique Métier isolée : Le GetFilteredLaunchesUseCase intervient à l'étape 8 pour transformer le flux de données brut en un flux trié/filtré selon les préférences de l'utilisateur.

Action de partage : Le geste utilisateur est traité en fin de flux, déclenchant une interaction avec le système Android (Intent).

8.2 Résilience : Scénario Offline avec Fallback

Ce flux illustre la gestion des erreurs réseau et la persistance.



NetworkMonitor : Le système utilise un ConnectivityManager (encapsulé dans NetworkMonitor) qui émet un flux booléen. Le ViewModel réagit instantanément à ce flux.

Résilience : En mode offline, l'application ne bloque pas l'utilisateur sur un écran d'erreur. Elle effectue un fallback sur la base de données locale (étape 4), permettant la consultation des données précédemment chargées.

Synchronisation Automatique : Le passage de Offline à Online déclenche automatiquement un refreshFromRemote(). C'est l'avantage de l'architecture réactive : l'utilisateur n'a pas besoin de "tirer pour rafraîchir", l'app se synchronise dès que possible.

Chapitre 9

Architecture et Fonctionnement

L'application SpaceX repose sur une architecture logicielle moderne respectant les principes de la Clean Architecture et du pattern MVVM (Model-View-ViewModel). Ce choix garantit une séparation stricte des responsabilités, facilitant ainsi la maintenance et l'évolution du code.

9.1 Organisation en couches

Le code est segmenté en trois couches distinctes :

A. Couche Présentation (UI) :
Cette couche gère l'interface utilisateur et l'état de l'écran.

— **Composables (Jetpack Compose)** : La hiérarchie est décomposée pour favoriser la réutilisation. SpaceXScreen et FavoritesScreen ne gèrent pas directement l'affichage des items ; ils délèguent cette tâche à LaunchListContent, qui lui-même utilise LaunchCard.

— **Adaptabilité** : L'attribut isLandscape dans les composants de liste montre la gestion native de l'orientation (Portrait/Paysage).

— **ViewModel** : Le LaunchesViewModel sert de pont. Il transforme les données brutes du Repository en StateFlow, un flux d'état réactif que Compose observe pour mettre à jour l'UI en temps réel.

B. Couche Domaine (Métier) :

C'est le cœur logique de l'application, indépendant des frameworks Android. Elle contient nos modèles métier (Launch) et nos Use Cases.

— **Use Cases (Interactors)** : Ils isolent des fragments de logique réutilisables. Par exemple, GetFilteredLaunchesUseCase contient l'algorithme complexe de tri et de filtrage, évitant ainsi de surcharger le ViewModel et permettant des tests unitaires isolés.

— **Modèles de domaine** : La classe Launch est une version "pure" des données, débarrassée des annotations de base de données ou de réseau.

C. Couche Data (Infrastructure) :

Cette couche gère la provenance des données.

— **Pattern Repository** : Le LaunchRepository agit comme un médiateur. Il implémente la stratégie Single Source of Truth (SSOT) : les données sont récupérées via SpaceXApi (Retrofit), sauvegardées dans LaunchDao (Room), et l'interface n'observe que la base de données locale.

— **Reactivité Flow** : Tous les retours de la base de données sont de type Flow<List<LaunchEntity» Cela signifie que si un lancement est marqué en favori, la base de données émet automatiquement une nouvelle liste, et l'interface se met à jour instantanément sans action manuelle (programmation réactive).

— **Persistance des préférences** : Le PreferencesManager utilise Jetpack DataStore pour sauvegarder l'état des filtres de l'utilisateur de manière asynchrone et sécurisée.

9.2 Parcours d'un flux de données (Data Flow)

L'application suit un flux de données unidirectionnel et réactif. Voici le cheminement type d'une donnée, de l'espace jusqu'à l'écran :

Requête : Le LaunchRepository appelle l'API via Retrofit.

Persistance : Les données JSON reçues sont immédiatement converties et stockées dans la base de données Room (Source unique de vérité).

Réactivité : La base de données émet un flux (Flow) de données dès qu'une modification survient.

Transformation : Le Use Case intercepte ce flux pour appliquer les filtres de recherche et le tri chronologique.

Observation : Le ViewModel convertit ce Flow en un StateFlow (flux d'état).

Rendu : L'interface Compose observe cet état et se redessine automatiquement (recomposition).

API → Repository → Room → UseCases → ViewModel → UI

9.3 Librairies tierces

Librairies utilisées :

Librairie	Rôle	Justification
Jetpack Compose	Framework UI	Approche déclarative moderne permettant une gestion simplifiée des états et des interfaces complexes, notamment pour l'affichage de listes dynamiques.
Room	Persistance	Bibliothèque officielle Android basée sur SQLite, offrant une validation des requêtes à la compilation et un support natif de la programmation réactive via Flow.
Kotlin Coroutines	Asynchronisme	Permet l'exécution de tâches en arrière-plan sans bloquer le thread principal, garantissant une interface utilisateur fluide et réactive.

TABLE 9.1 – Librairies utilisées dans l'application SpaceX Tracker

9.4 Parc Matériel et Cible

L'application a été conçue pour couvrir une large base d'utilisateurs tout en utilisant les dernières fonctionnalités de sécurité et de performance d'Android.

API Minimum : 26 (Android 8.0 Oreo)

Pourquoi ? Pour supporter nativement les API de temps (java.time) nécessaires au formatage des dates de lancement sans librairies de rétro-compatibilité lourdes.

API Cible : 35 (Android 15)

Pourquoi ? Pour garantir la compatibilité avec les derniers standards de sécurité et optimisations énergétiques de Google en 2025.

Orientations : Support complet des modes Portrait et Paysage, avec une adaptation dynamique de la mise en page (utilisation de LaunchListContent adaptatif).

9.5 Éléments clés de compréhension

Gestion du Offline : Grâce au Repository et à Room, l'utilisateur a accès aux données précédemment chargées même sans connexion. Un NetworkMonitor prévient l'utilisateur en temps réel de l'état de sa connexion via une Snackbar.

Injection de dépendances : Bien que simplifiée via une ViewModelFactory, elle permet de découpler la création des objets (Repository, Use Cases) de leur utilisation, facilitant ainsi les futurs tests ou le passage à un framework comme Hilt.

Note sur la performance : L'utilisation de collectAsStateWithLifecycle dans les écrans garantit que les flux de données s'arrêtent automatiquement lorsque l'application est en arrière-plan, économisant ainsi la batterie de l'appareil.

Chapitre 10

Améliorations et Perspectives

— Enrichir l'application avec d'autres fonctionnalités
— Pagination avec Paging 3
— Internationalisation multilingue

Chapitre 11

Conclusion

Le développement de SpaceX Tracker a permis d'appliquer les standards modernes du développement Android. L'utilisation de Kotlin Coroutines, de Room et de Jetpack Compose garantit une application robuste, performante et maintenable.

Chapitre 12

Bibliographie / Webographie

— API SpaceX : `https://github.com/r-spacex/SpaceX-API`
— Docs Retrofit : `https://square.github.io/retrofit/`
— Docs Moshi : `https://github.com/square/moshi`
— Docs Coil : `https://coil-kt.github.io/coil/`
— Jetpack Compose : `https://developer.android.com/jetpack/compose`
— Outil Diagrammes : `mermaid.live`